

# Solving MDPs: Policy Iteration

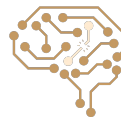
5 Tir 1405

Arash Maroriad



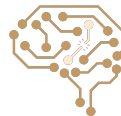
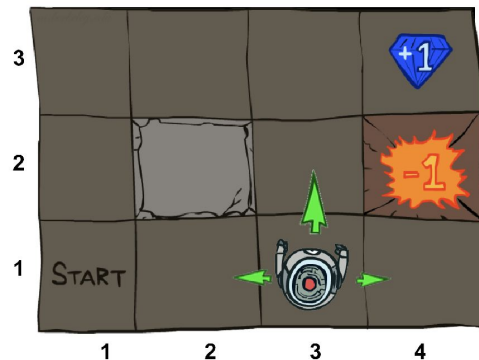
CE 417-Artificial Intelligence, Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188 & Sharif UT CE417



# Markov Decision Processes (MDP)

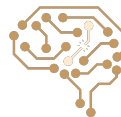
- An MDP is defined by:
  - A set of states  $s \in S$
  - A set of actions  $a \in A$
  - A transition function  $T(s, a, s')$ 
    - Probability that  $a$  from  $s$  leads to  $s'$ , i.e.,  $P(s' | s, a)$
    - Also called the model or the dynamics
  - A reward function  $R(s, a, s')$ 
    - Sometimes just  $R(s)$  or  $R(s')$
  - A start state
  - Maybe a terminal state



# Formulation: What is the purpose of the agent?

maximize it's reward across all timesteps (**Return/Utility**):

$$U([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + R(s_2, a_2, s_3) + \dots$$



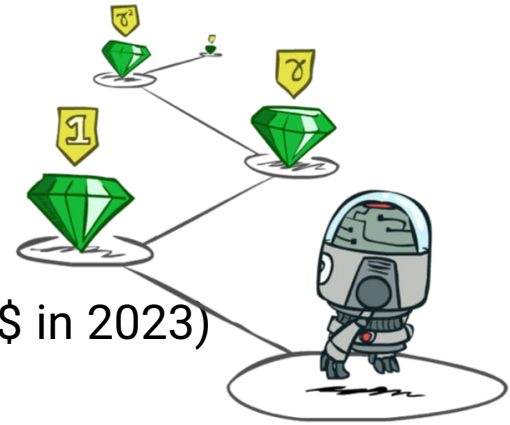
# Finite Horizons and Discounting

## ► Problems with Utility Function:

- ☐ obtaining infinite reward?
- ☐ prefer rewards now to rewards later (1\$ in 2000 > 1\$ in 2023)

## ► Solutions:

- ☐ finite horizons: defines a "lifetime" for agents
- ☐ discount factors: additive utility -> discounted utility (get rid of infinite utility)

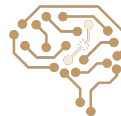


$$U([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$$



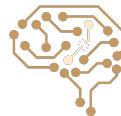
# Optimal Policy

- For MDPs, we want an optimal **policy**  $\pi^*: S \rightarrow A$ 
  - A policy  $\pi$  gives an action for each state
  - An optimal policy is one that **maximizes expected utility** if followed
- Expectimax didn't compute entire policies
  - It computed the action for a single state only



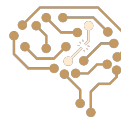
# How To Find Optimal Policy in MDP?

- Two Iterative Algorithms:
  - **Value Iteration**
  - **Policy Iteration**
- First, let's define some notations ...



# Some New Quantities

- $V^\pi(\mathbf{s})$  = expected return starting from  $\mathbf{s}$  and acting based on policy  $\pi$
- $V^*(\mathbf{s})$  = expected return starting from  $\mathbf{s}$  and acting optimally ( $\pi^*$ )
- $Q^\pi(\mathbf{s}, \mathbf{a})$  = expected return starting from  $\mathbf{s}$  and choosing action  $\mathbf{a}$ , then acting based on policy  $\pi$
- $Q^*(\mathbf{s}, \mathbf{a})$  = expected return starting from  $\mathbf{s}$  and choosing action  $\mathbf{a}$ , then acting optimally ( $\pi^*$ )
- Why expected? Due to the randomness, when acting based on  $\pi$  or  $\pi^*$ , we may experience different paths, so we must make an expectation

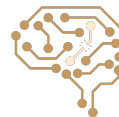




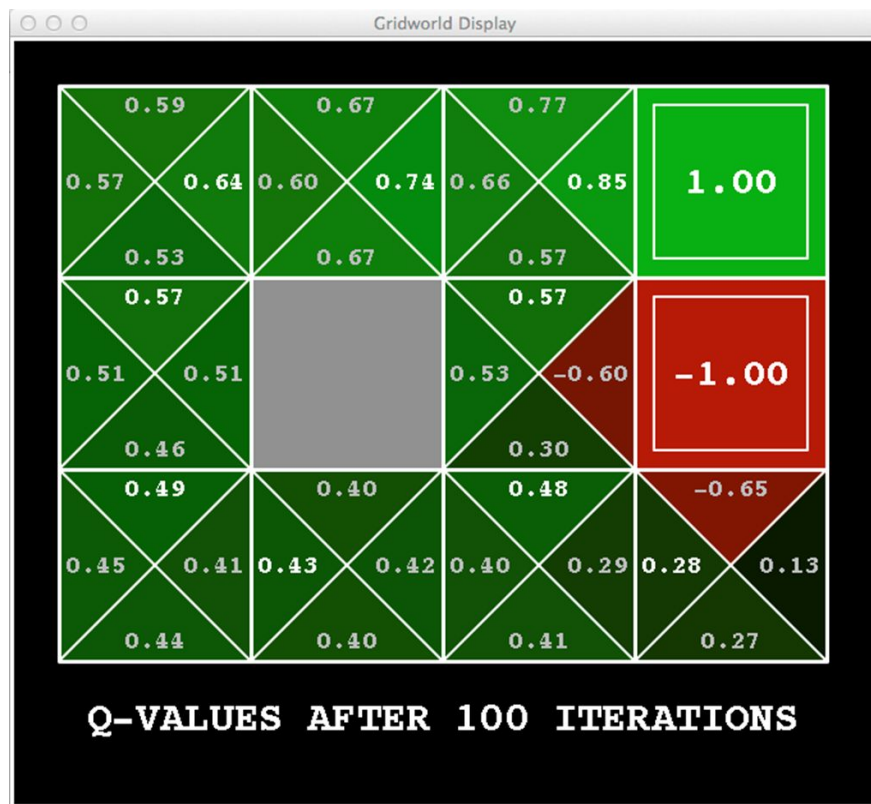
# Example: Optimal Values



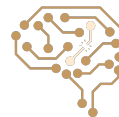
Noise = 0.2  
Discount = 0.9  
Living reward = 0



# Example: Optimal Q-Values



Noise = 0.2  
Discount = 0.9  
Living reward = 0

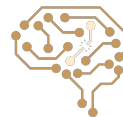
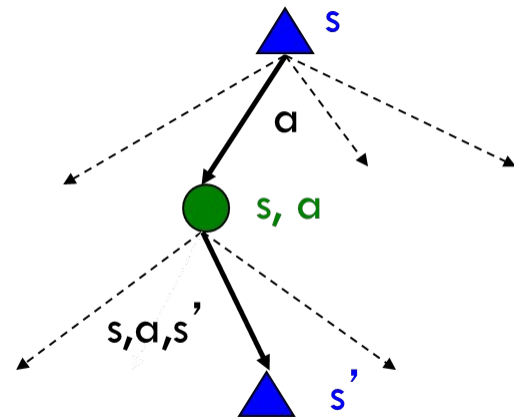


# The Bellman Optimality Equations

$$V^*(s) = \max_a Q^*(s, a)$$

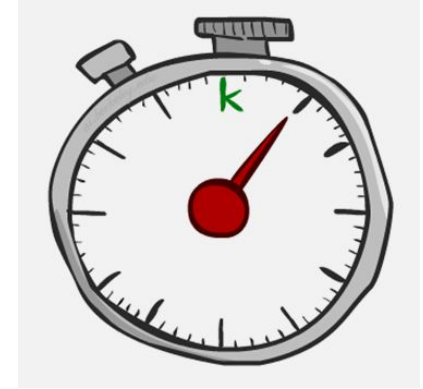
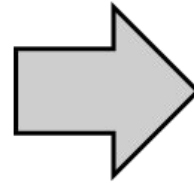
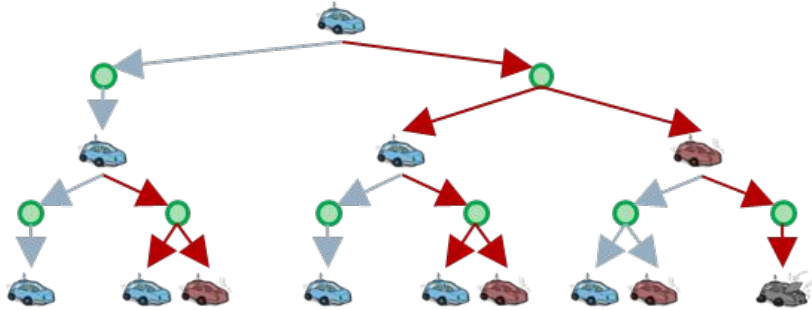
$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

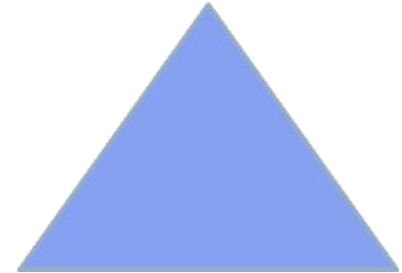


# Time-Limited Values

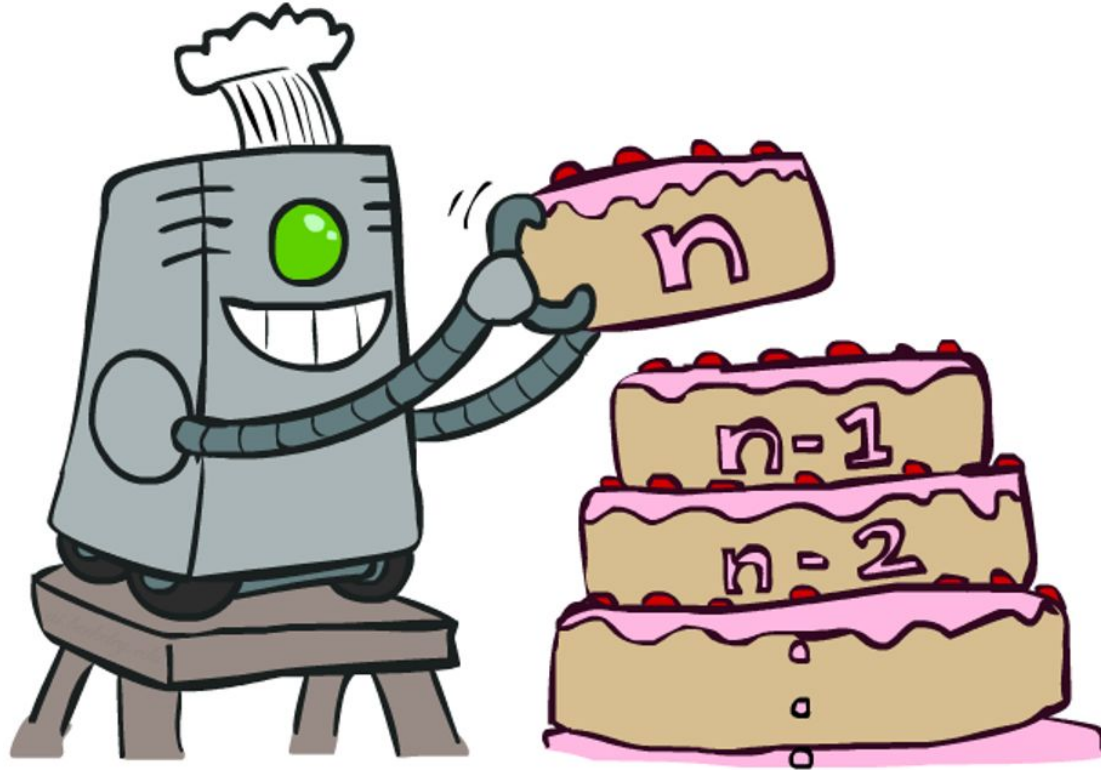
- Define  $V_k(s)$  to be the optimal value of  $s$  if the game ends in  $k$  more time steps



$V_2(\text{car})$



# Value Iteration Algorithm



# Value Iteration

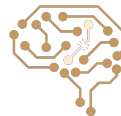
- Bellman equations **characterize** the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

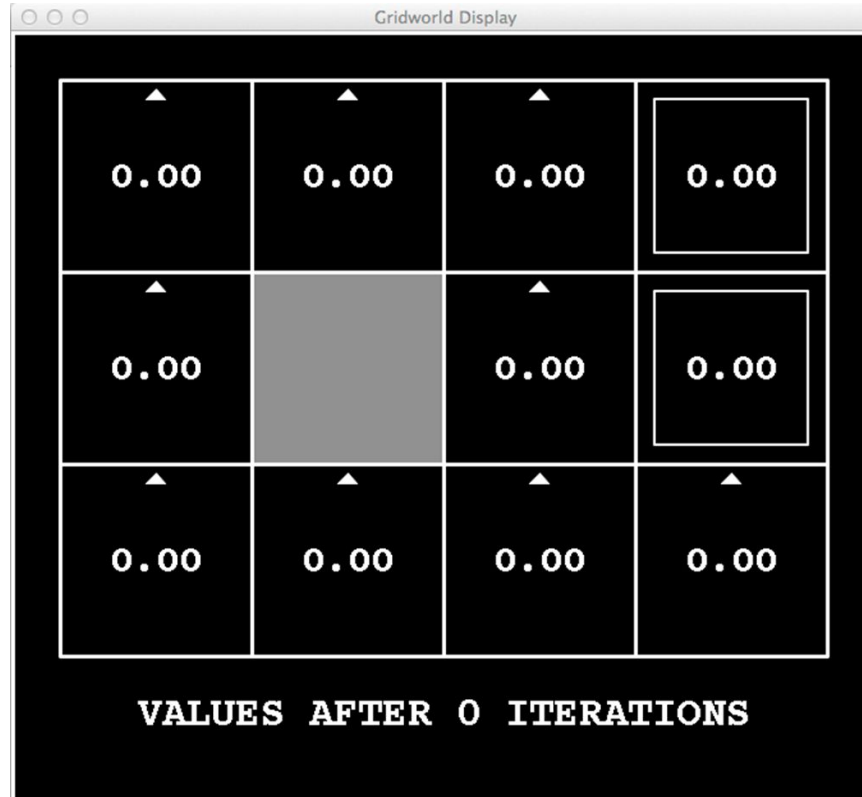
- Start with  $V_0(s) = 0$ : no time steps left means an expected reward sum of zero
- Repeat until convergence

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Complexity of each iteration:  $O(S^2A)$



# Value Iteration: $k=0$

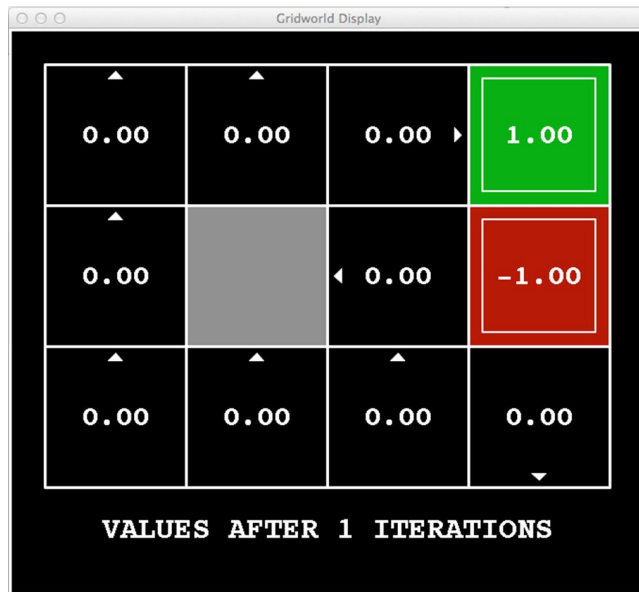


Noise = 0.2  
Discount = 0.9  
Living reward = 0

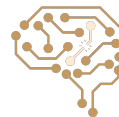


# Value Iteration: k=1

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



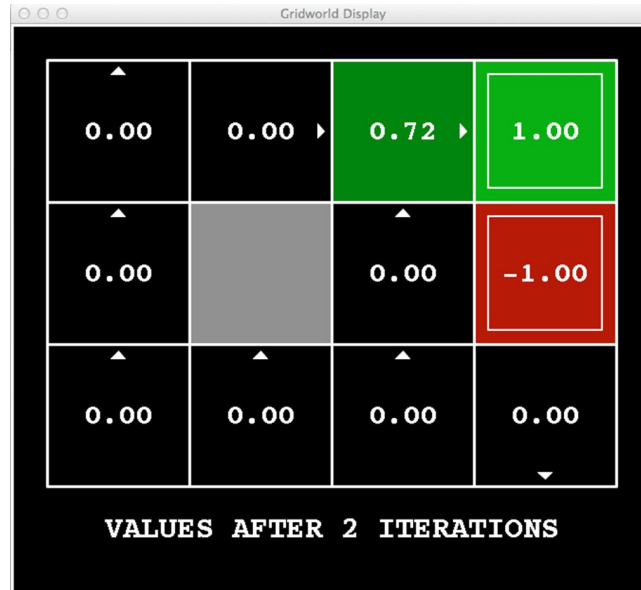
Noise = 0.2  
Discount = 0.9  
Living reward = 0





# Value Iteration: k=2

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

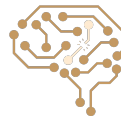


# Value Iteration: k=3

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)(R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

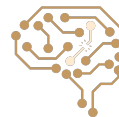


# Value Iteration: k=4

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)(R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

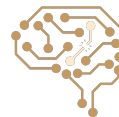


# Value Iteration: k=5

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)(R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

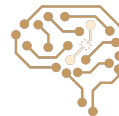


# Value Iteration: k=6

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

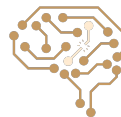


# Value Iteration: k=7

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)(R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0



# Value Iteration: k=8

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

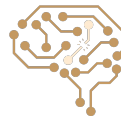


# Value Iteration: k=9

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)(R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0





# Value Iteration: k=10

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



Noise = 0.2  
Discount = 0.9  
Living reward = 0

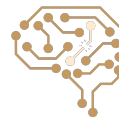


# Value Iteration: k=100

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s'))$$



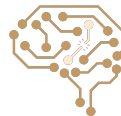
Noise = 0.2  
Discount = 0.9  
Living reward = 0



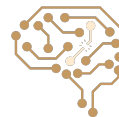
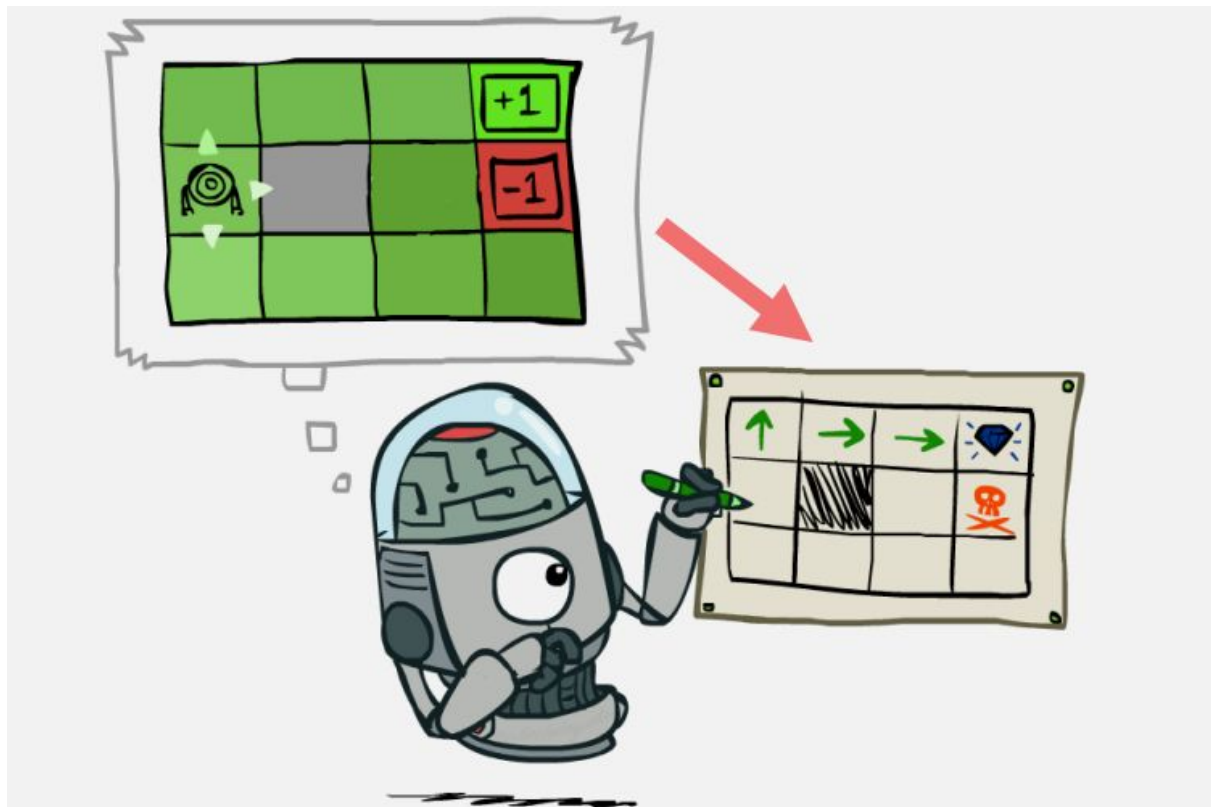
# Value Iteration Convergence

**Theorem.** Value iteration converges. At convergence, we have found the optimal value function  $V^*$  for the discounted infinite horizon problem, which satisfies the Bellman equations

$$\forall S \in S : \quad V^*(s) = \max_A \sum_{s'} T(s, a, s') \left[ R(s, a, s') + \gamma V^*(s') \right]$$

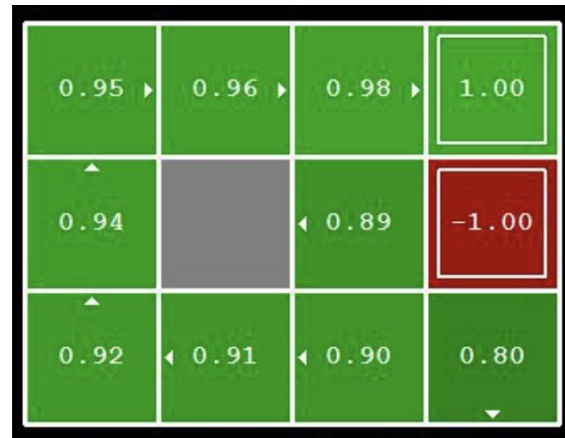


# Policy Extraction



# Policy Extraction: Computing Actions from Values

- Let's imagine we have the optimal values  $V^*(s)$
- How should we act?
  - It's not obvious!
- We need to do a mini-expectimax (one step)



$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

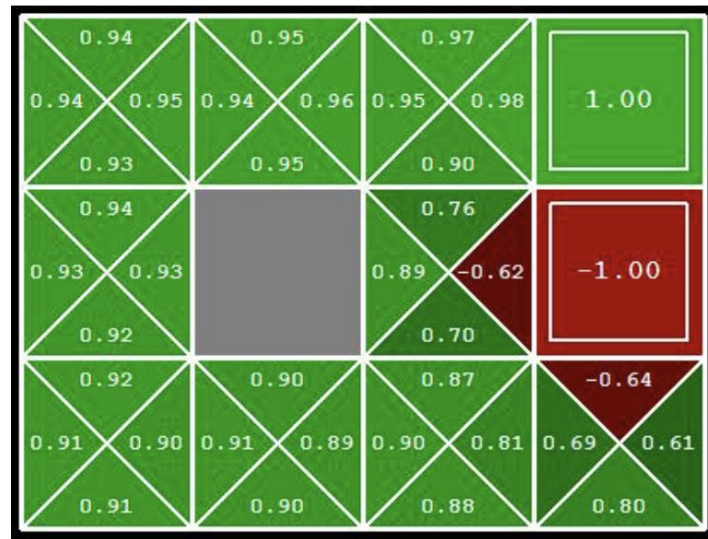
- This is called **policy extraction**, since it gets the policy implied by the values



# Policy Extraction: Computing Actions from Q-Values

- Let's imagine we have the optimal q-values:
- How should we act?
  - Completely trivial to decide!

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$



- Important lesson: actions are easier to extract from q-values than values!



---

**Algorithm 2:** Value Iteration Algorithm

---

**Data:**  $\theta$ : a small number

**Result:**  $\pi$ : a deterministic policy s.t.  $\pi \approx \pi_*$

**Function** *ValueIteration* **is**

```
/* Initialization                                     */
Initialize  $V(s)$  arbitrarily, except  $V(\text{terminal})$ ;
 $V(\text{terminal}) \leftarrow 0$ ;
/* Loop until convergence                             */
 $\Delta \leftarrow 0$ ;
while  $\Delta < \theta$  do
    for each  $s \in S$  do
         $v \leftarrow V(s)$ ;
         $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ ;
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ ;
    end
end
/* Return optimal policy                               */
return  $\pi$  s.t.  $\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ ;
```

**end**

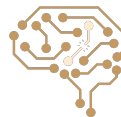


# Problems with Value Iteration

- Value iteration repeats the Bellman updates:

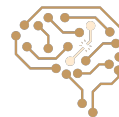
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Problem 1: It's slow –  $O(S^2A)$  per iteration
- Problem 2: The “max” at each state rarely changes
- Problem 3: The policy often converges long before the values

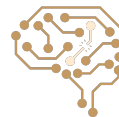
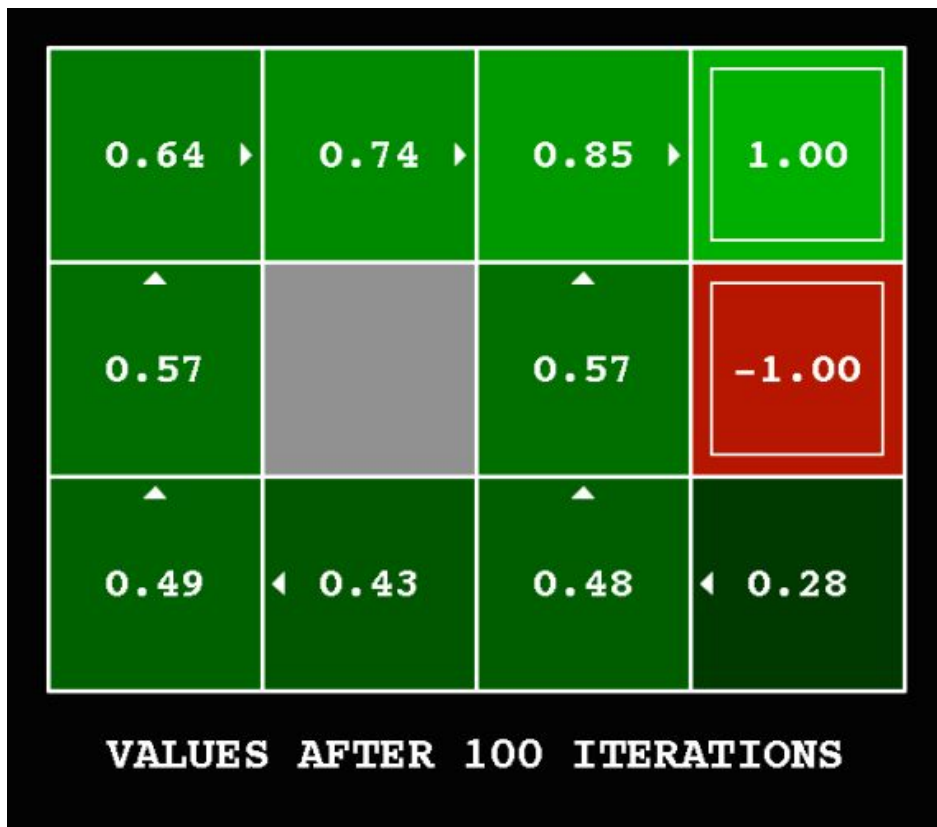




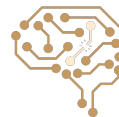
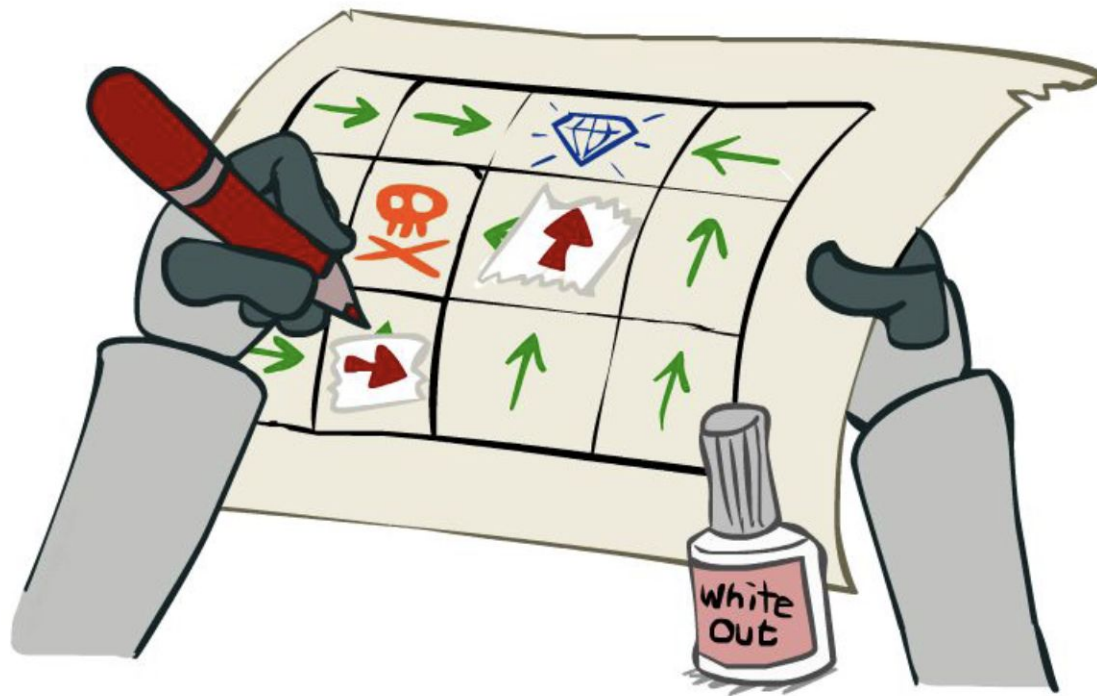
## Value Iteration: k=12



## Value Iteration: k=100



# Policy Iteration Algorithm



# Policy Iteration Algorithm

- Alternative approach for optimal values:
  - **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence
  - **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
  - Repeat steps until policy converges
- This is **policy iteration**
  - It's still optimal!
  - Can converge (much) faster under some conditions



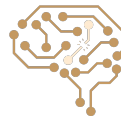
# Policy Evaluation: Finding State Values Under a Fixed Policy

- Another basic operation: compute the utility of a state  $s$  under a fixed (generally non-optimal) policy
- Define the utility of a state  $s$ , under a fixed policy  $\pi$ :  
 $V^\pi(s)$  = expected total discounted rewards starting in  $s$  and following  $\pi$
- Recursive relation (one-step look-ahead / Bellman equation):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Hint: recall Bellman equation for optimal policy:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



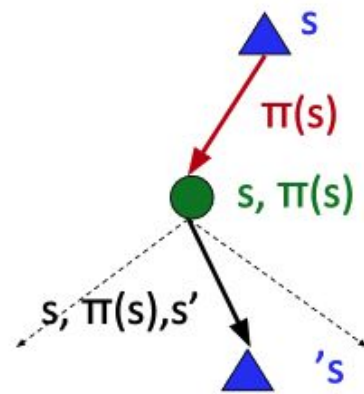
# Policy Evaluation Algorithm

- How do we calculate the  $V$ 's for a fixed policy  $\pi$ ?
- Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

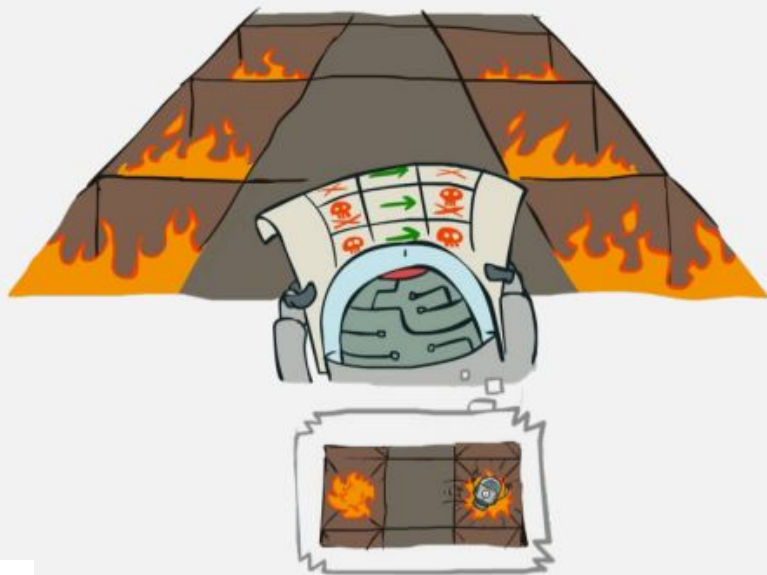
$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency:  $O(S^2)$  per iteration
- Idea 2: Without the maxes, the Bellman equations are just a linear system
  - Solve with your favorite linear system solver

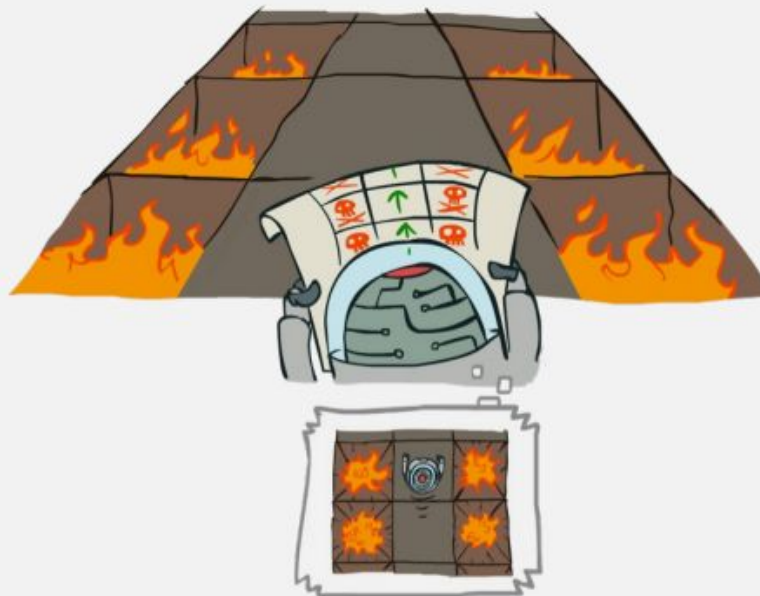


# Example: Policy Evaluation

Always Go Right



Always Go Forward

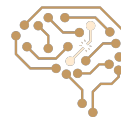


# Example: Policy Evaluation

Always Go Right



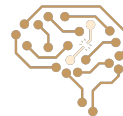
Always Go Forward





# Policy Improvement

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$



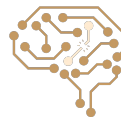
# Policy Iteration

- **Evaluation:** For fixed current policy  $\pi$ , find values with policy evaluation:
  - Iterate until values converge (might run this for many iterations):

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- **Improvement:** For fixed values, get a better policy using policy extraction
  - One-step look-ahead (run only once):

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$



**Algorithm 1:** Policy Iteration Algorithm**Data:**  $\theta$ : a small number**Result:**  $V$ : a value function s.t.  $V \approx v_*$ ,  $\pi$ : a deterministic policy  
s.t.  $\pi \approx \pi_*$ **Function** *PolicyIteration* **is**

```

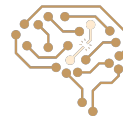
/* Initialization */
Initialize  $V(s)$  arbitrarily;
Randomly initialize policy  $\pi(s)$ ;
/* Policy Evaluation */
 $\Delta \leftarrow 0$ ;
while  $\Delta < \theta$  do
    for each  $s \in S$  do
         $v \leftarrow V(s)$ ;
         $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))[r + \gamma V(s')]$ ;
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ ;
    end
end
/* Policy Improvement */
policy-stable  $\leftarrow true$ ;
for each  $s \in S$  do
    old-action  $\leftarrow \pi(s)$ ;
     $\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$ ;
    if old-action  $\neq \pi(s)$  then
        policy-stable  $\leftarrow false$ ;
    end
end
if policy-stable then
    return  $V \approx v_*$  and  $\pi \approx \pi_*$ ;
else
    go to Policy Evaluation;
end
end

```



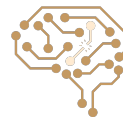
# Policy Iteration Convergence

**Theorem.** Policy iteration is guaranteed to converge and at convergence, the current policy and its value function are the optimal policy and the optimal value function!



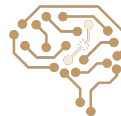
# Comparison

| Policy Iteration                                                                                                                                          | Value Iteration                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Starts with a random policy<br>Algorithm is more complex<br>Guaranteed to converge<br>Cheaper to compute<br>Requires few iterations to converge<br>Faster | Starts with a random value function<br>Algorithm is simpler<br>Guaranteed to converge<br>More expensive to compute<br>Requires more iterations to converge<br>Slower |



# Summary: MDP Algorithms

- So you want to....
  - Find optimal policy: use value iteration or policy iteration
  - Compute values for a particular policy: use policy evaluation
  - Turn your values into a policy: use policy extraction (one-step lookahead)
- Next Step: **RL**, When You Do Not Know About Transition Probability and Reward!



# Reinforcement Learning

- Still assume a Markov decision process (MDP):

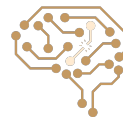
- A set of states  $s \in S$
- A set of actions (per state)  $A$
- A model  $T(s,a,s')$
- A reward function  $R(s,a,s')$

- Still looking for a policy  $\pi(s)$



- New twist: don't know  $T$  or  $R$

- I.e. we don't know which states are good or what the actions do
- Must try out actions and states to learn
  - Q1: How to learn from things tried? (today, Passive Reinforcement Learning)
  - Q2: What to decide to try? (Thursday, Active Reinforcement Learning)



# Next Session: Reinforcement Learning (RL)

